Análisis Estructural y Protocolo de Migración Transdimensional de MambaVision: De PyTorch a Keras 3

Fundamentos Teóricos y Evolución de la Arquitectura MambaVision

El modelado secuencial y espacial en el aprendizaje profundo ha experimentado un cambio de paradigma fenomenal con la introducción de los Modelos de Espacio de Estados (SSMs, por sus siglas en inglés). Originalmente concebidos para procesar señales unidimensionales de tiempo continuo y posteriormente adaptados para el procesamiento de lenguaje natural a través de arquitecturas como S4 y el modelo Mamba original, los SSMs ofrecen una complejidad computacional lineal $\mathcal{O}(n)$ respecto a la longitud de la secuencia, superando la complejidad cuadrática $\mathcal{O}(n^2)$ que paraliza a los Transformers tradicionales en contextos de ventanas de contexto largas.[1] La formulación matemática fundacional de estos modelos describe un sistema físico mediante variables de estado ocultas $h(t)$, procesadas mediante una matriz de transición A que obliga a la estabilidad mediante valores propios negativos, y matrices de proyección B y C que asimilan la entrada y emiten la salida, respectivamente.[2] Sin embargo, la migración de este paradigma estrictamente secuencial y causal al dominio del procesamiento de imágenes, donde la dependencia espacial es bidireccional y bidimensional, presentó un desafío computacional significativo.

Presentado en la prestigiosa conferencia CVPR 2025 por investigadores de NVIDIA, MambaVision emerge como una solución arquitectónica híbrida de vanguardia que reescribe la aplicabilidad de los SSMs para la visión por computadora.[5] A diferencia de iteraciones previas como Vision Mamba (Vim), que dependían de un procesamiento bidireccional exhaustivo que ralentizaba la inferencia y el entrenamiento debido a la sobrecarga computacional de procesar secuencias hacia adelante y hacia atrás de manera redundante [8], MambaVision propone un diseño jerárquico finamente calibrado. Esta arquitectura extrae características semánticas a través de cuatro etapas progresivas, fusionando la extracción de características de alta frecuencia mediante redes neuronales convolucionales (CNN) en las etapas tempranas, y delegando el razonamiento global a una amalgama de bloques Mamba modificados y bloques de autoatención (Transformers) en las etapas profundas.[9]

El impacto empírico de esta hibridación ha establecido una nueva frontera de Pareto en el conjunto de datos ImageNet-1K, logrando un equilibrio sin precedentes entre la exactitud

(Top-1 accuracy) y el rendimiento de procesamiento de imágenes (throughput).[10] Modelos a escala como MambaVision-L-21K demuestran esta superioridad al alcanzar una precisión del 88.1% tras un preentrenamiento en ImageNet-21K y ajuste fino en ImageNet-1K, con 227.9 millones de parámetros y 34.9 GFLOPs a una resolución de 224×224.[6] Adicionalmente, el modelo supera a arquitecturas equivalentes en tareas de procesamiento en sentido descendente (downstream), como la detección de objetos y la segmentación semántica en los conjuntos de datos MS COCO y ADE20K, cimentando su estatus como un extractor de características de propósito general excepcionalmente robusto.[10]

La implementación oficial de este modelo reside nativamente en PyTorch, utilizando abstracciones de nivel inferior para invocar núcleos CUDA altamente optimizados para la operación de escaneo selectivo (selective_scan_fn).[9] No obstante, los imperativos de despliegue industrial, la compilación de grafos computacionales para aceleradores tensores (TPUs) y la necesidad de agnosticismos de hardware requieren la transmutación de estos pesos hacia el ecosistema multi-backend de Keras 3.[15] Keras 3 permite ejecutar modelos sobre TensorFlow, JAX o PyTorch indistintamente, pero impone arquitecturas de tensores radicalmente diferentes. Este informe deconstruye el diccionario de estados (state_dict) de PyTorch, mapea su ontología tensorial y formula un protocolo de migración matemáticamente riguroso hacia Keras 3.

Disección Anatómica del state_dict en PyTorch

El mecanismo de serialización y preservación de estado en PyTorch consolida los tensores multidimensionales aprendibles (pesos, sesgos) y las estadísticas móviles continuas (medias y varianzas de normalización de lotes) en un diccionario ordenado de Python referenciado como state_dict.[16] En MambaVision, este diccionario encapsula una topología jerárquica densa. La invocación de torch.load revela que los puntos de control frecuentemente albergan metadatos externos y anomalías de nomenclatura dependientes del contexto de entrenamiento original. Por ejemplo, los regímenes de Entrenamiento Distribuido de Datos (DDP) a menudo anteponen el prefijo module. a todas las claves, mientras que ciertas abstracciones de extracción de características anteponen el prefijo encoder..[17] El código base oficial incluye rutinas heurísticas para limpiar estos artefactos, despojando a las claves de los primeros siete caracteres si detectan la presencia paralela, o reemplazando subcadenas específicas.[17]

Una vez esterilizado de artefactos de entrenamiento transitorios, el diccionario expone una taxonomía de capas intrincada que mapea directamente la concepción teórica de la red. Para facilitar una comprensión exhaustiva de los prefijos reales y los tensores alojados, la arquitectura se divide analíticamente en cinco componentes cardinales: la incrustación de parches (Stem/PatchEmbed), las etapas de submuestreo espacial (Downsample), los bloques convolucionales de extracción temprana (ConvBlocks), los bloques de mezcla jerárquica (MambaVisionMixer y TransformerBlock), y el cabezal clasificador global (Head).

Módulo de Incrustación de Parches (PatchEmbed)

El procesamiento inicial de cualquier tensor de imagen tridimensional de entrada $\mathbb{R}^{B \times C \times H \times W}$ requiere una proyección a un espacio de incrustación de alta dimensión y una subsiguiente o simultánea reducción de la resolución espacial. En MambaVision, este proceso no se delega a una única convolución agresiva (como ocurre en el proyector clásico de los Vision Transformers), sino a un submódulo secuencial compuesto denominado PatchEmbed, que ejecuta dos convoluciones superpuestas intercaladas con normalización y activaciones no lineales.[17]

La lógica subyacente de este enfoque dual es minimizar la pérdida abrupta de información estructural de alta frecuencia, un defecto bien documentado en las proyecciones de un solo paso. El bloque de código de PyTorch encapsula estas operaciones en una variable nn.Sequential llamada conv_down.[17] La proyección semántica abstracta proj, a menudo invocada para homogeneizar interfaces con arquitecturas preexistentes, se define matemáticamente como un mapeo de identidad (nn.Identity()), por lo que no genera vectores de estado.[17] La estructura precisa extraída del state_dict se ilustra en la tabla adjunta.

Prefijo Exacto del state_dict PyTorch	**Instanciación de Clase Subyacente**	**Función Matemática y Dimensionalidad**
patch_embed.conv_down.0.weight	nn.Conv2d ($3 \times$, stride 2, padding 1)	Proyección inicial del espacio de color a la dimensión intermedia. Carece de tensor de sesgo (bias=False).
patch_embed.conv_down.1.weight	nn.BatchNorm2d (γ)	Parámetro de escalado afín para la normalización espacial intermedia.
patch_embed.conv_down.1.bias	nn.BatchNorm2d (β)	Parámetro de desplazamiento afín.
patch_embed.conv_down.1.running_mean	nn.BatchNorm2d (μ)	Estadística móvil de media inferida durante el entrenamiento.
patch_embed.conv_down.1.running_var	nn.BatchNorm2d (σ^2)	Estadística móvil de varianza inferida durante el

		entrenamiento.
patch_embed.conv_down.3 .weight	nn.Conv2d ($3 \times$, stride 2, padding 1)	Extensión dimensional secundaria a la capacidad completa dim. Sin sesgo.
patch_embed.conv_down.4 .weight	nn.BatchNorm2d (γ)	Parámetro de escalado de la segunda etapa de normalización.
patch_embed.conv_down.4 .bias	nn.BatchNorm2d (β)	Parámetro de desplazamiento de la segunda etapa de normalización.
patch_embed.conv_down.4 .running_mean	nn.BatchNorm2d (μ)	Estadística móvil de la media final del incrustador.
patch_embed.conv_down.4 .running_var	nn.BatchNorm2d (σ^2)	Estadística móvil de la varianza final del incrustador.

El escrutinio de estas claves revela que el módulo de incrustación prioriza la normalización estricta sobre la aplicación de activaciones afines con sesgo. Los descriptores numéricos discretos (0, 1, 3, 4) reflejan los índices internos posicionales generados implícitamente por el contenedor nn.Sequential de PyTorch al compilar la gráfica, donde el índice 2 corresponde al operador inyectivo de rectificación (ReLU), el cual no posee componentes paramétricos.

Módulos de Transición y Submuestreo Espacial (Downsample)

A lo largo de las cuatro macroetapas jerárquicas del modelo, la resolución espacial de los mapas de características se contrae, duplicando en contrapartida la dimensionalidad de los canales semánticos. Las fronteras entre estas etapas están interconectadas mediante el módulo Downsample.[9] El código fuente define una función de reducción que opera de manera determinista utilizando una operación convolucional con tamaño de núcleo $3 \times$, un paso iterativo de 2 y un relleno perimetral de 1.[17]

Al iterar sobre la estructura del diccionario, estos módulos residen semánticamente en el mismo nivel jerárquico que los bloques generativos, usualmente integrados dentro de la definición paramétrica de los levels. Sin embargo, las iteraciones de la red que no aplican

contracción condicional omiten este bloque instanciándolo como tipo nulo.[18] La parametrización concreta se delega a un submódulo reduction.

Prefijo Exacto del state_dict PyTorch	Instanciación de Clase Subyacente	Función Matemática y Dimensionalidad
levels.[i].downsample.reduction.0.weight	nn.Conv2d ($3 \times$, stride 2, padding 1)	Tensor principal que efectúa la transmutación multiescala inter-capa. Carece de parámetros de sesgo debido a normativas arquitectónicas posteriores (bias=False). El índice [i] oscila de 0 a 2, indicando la etapa precursora.

Etapas Tempranas de Extracción: Bloques Convolucionales (ConvBlock)

La divergencia más pronunciada entre MambaVision y los enfoques puramente Transformer radica en la comprensión de que las etapas incipientes operan sobre mapas de características de alta densidad espacial, donde la autoatención impone cuellos de botella intratables debido a su escalamiento cuadrático $\mathcal{O}(N^2)$ donde $N = H \times W$.[9] En consecuencia, MambaVision implementa módulos ConvBlock residuales en sus dos primeros niveles (etapa 1 y etapa 2).[9]

El flujo lógico del código oficial especifica la creación de un nn.ModuleList bajo el nombre self.levels. Una condición determinista (conv = True if (i == 0 or i == 1) else False) dirige al enrutador de red para instanciar bloques estrictamente convolucionales en estas iteraciones.[17] Cada bloque encapsula un camino convolucional de dos fases. La primera iteración expande o procesa la representación utilizando un núcleo de convolución espacial de vecindad $3 \times$, y es estabilizada mediante una normalización de lote acoplada a una activación no lineal Gaussiana aproximada (GELU). Un segundo paso homólogo cierra el ciclo antes de la suma residual integrativa.[17]

Prefijo Exacto del state_dict PyTorch	Instanciación de Clase Subyacente	Función Matemática y Dimensionalidad
levels.[i].blocks.[j].conv1.weight	nn.Conv2d ($3 \times$, stride 1,	Extractor inicial de características locales en el

	padding 1)	bloque residual.
levels.[i].blocks.[j].conv1.bias	Parámetro escalar unidimensional	Desplazamiento aditivo posterior a la convolución.
levels.[i].blocks.[j].norm1.weight	nn.BatchNorm2d (γ)	Ponderador de escalamiento dinámico post-activación preliminar.
levels.[i].blocks.[j].norm1.bias	nn.BatchNorm2d (β)	Ponderador de desplazamiento dinámico de la representación.
levels.[i].blocks.[j].norm1.running_mean	nn.BatchNorm2d (μ)	Traza estadística de las activaciones pre-normalizadas.
levels.[i].blocks.[j].norm1.running_var	nn.BatchNorm2d (σ^2)	Traza de la dispersión variacional de la distribución latente.
levels.[i].blocks.[j].conv2.weight	nn.Conv2d ($3 \times$, stride 1, padding 1)	Segunda proyección espacial del bloque, completando la función del receptor local.
levels.[i].blocks.[j].conv2.bias	Parámetro escalar unidimensional	Vector de sesgo afín de la segunda convolución.
levels.[i].blocks.[j].norm2.weight	nn.BatchNorm2d (γ)	Ponderador de escalamiento previo a la adición del atajo residual.
levels.[i].blocks.[j].norm2.bias	nn.BatchNorm2d (β)	Desplazador afín correspondiente a la fase terminal del bloque.
levels.[i].blocks.[j].norm2.running_mean	nn.BatchNorm2d (μ)	Estadística de media correspondiente a la rama

		convolucional de salida.
levels.[i].blocks.[j].norm2.running_var	nn.BatchNorm2d (σ^2)	Estadística de varianza retenida durante los pases de optimización.

En esta fase de extracción arquitectónica, el identificador [i] pertenece al conjunto $\{0, 1\}$ denotando las etapas iniciales, mientras que [j] representa la profundidad secuencial del bloque convolucional dentro de la etapa correspondiente, guiado por la variable de configuración depths.[17]

Etapas Híbridas Profundas: MambaVisionMixer y TransformerBlock

La innovación cardinal del modelo reside en la arquitectura de las etapas 3 y 4. Al haber mitigado la resolución espacial a hiper-dimensiones computacionalmente asimilables, el diseño amalgama el modelado secuencial lineal de Mamba con la capacidad auto-atencional canónica del Transformer.[9] De forma determinista, dado un conteo de capas N, las primeras $N/2$ capas emplean la lógica MambaVisionMixer, mientras que las subsiguientes $N/2$ aplican el bloque genérico de TransformerBlock.[9] Esta hibridación supera empíricamente el rendimiento de los bloques puros, equilibrando la retención de dependencia temporal de los SSMs discretizados y el alcance sináptico iterativo de las consultas, claves y valores atencionales.

Escrutinio del Módulo MambaVisionMixer

El bloque MambaVisionMixer es un rediseño de la infraestructura original de Mamba ajustado para procesar un espacio latente de imágenes carente de asimetría direccional causal. En la teoría original de Mamba en el procesamiento del lenguaje natural, las activaciones se evaluaban restrictivamente sobre horizontes pasados.[4] MambaVision altera este paradigma aplicando transformaciones convolucionales con extensiones espaciales reflectivas y simétricas.[10]

La inspección del algoritmo de pseudo-código proporciona un plano anatómico perfecto de la inyección paramétrica generada en PyTorch.[9] El flujo se inicia con un mapeo lineal in_proj que expande el espacio vectorial de entrada por un factor multiplicativo (frecuentemente 2) y divide la topología en dos ramas funcionales independientes: la rama x y la rama de control z.[9] Ambos tensores atraviesan independientemente proyecciones convolucionales espaciales no causales, etiquetadas en el núcleo como conv1d_x y conv1d_z, respectivamente.[9]

La rama principal procede hacia una proyección densa de dependencia selectiva x_proj, el

núcleo absoluto de la operación Mamba, emitiendo dinámicamente los parámetros latentes del espacio temporal continuo (los desplazamientos iterativos Δ, y las matrices condicionales B y C) en base al contenido de la imagen de entrada.[4] Para estabilizar el operador de transición continuo intrínseco de los SSMs, la matriz central del sistema A no se formula como un hiperparámetro no dinámico lineal, sino que es calculada a partir de su logaritmo paramétrico parametrizado estáticamente como el tensor libre de decaimiento escalar A_log.[4] A continuación se catalogan exhaustivamente los parámetros de este subsistema.

Prefijo Exacto del state_dict PyTorch	**Instanciación de Clase Subyacente**	**Función Matemática y Dimensionalidad**
levels.[i].blocks.[j].mixer.in_proj.weight	nn.Linear (Dense)	Expansor inicial paramétrico. Mapea el flujo unidimensional original a dos hemisferios expansivos de procesamiento.
levels.[i].blocks.[j].mixer.in_proj.bias	Tensor unidimensional	Ponderación de desplazamiento inicial.
levels.[i].blocks.[j].mixer.conv1d_x.weight	nn.Conv1d ($K =$, padding 'same', grupos)	Modelado causal de la dinámica espacial en la rama de información. Ejecutado como una convolución de profundidad 1D (grouped convolution).
levels.[i].blocks.[j].mixer.conv1d_x.bias	Tensor unidimensional	Sesgo aplicado a la ruta principal post-convolución.
levels.[i].blocks.[j].mixer.conv1d_z.weight	nn.Conv1d ($K =$, padding 'same', grupos)	Modelado de la compuerta analítica (gating branch) operada bajo un esquema simétrico al principal.
levels.[i].blocks.[j].mixer.conv1d_z.bias	Tensor unidimensional	Sesgo aplicativo de la ruta de compuerta.

levels.[i].blocks.[j].mixer.x_proj.weight	nn.Linear (Dense)	Reducción dimensional para emitir los selectores locales específicos Δ, B, C. Carece de desplazamiento (bias=False).
levels.[i].blocks.[j].mixer.dt_proj.weight	nn.Linear (Dense)	Escalamiento de la dimensión del horizonte de tiempo continuo Δ.
levels.[i].blocks.[j].mixer.dt_proj.bias	Tensor unidimensional	Inicializado mediante una distribución pseudo-aleatoria exponencial; no debe re-inicializarse bajo contingencias estocásticas ordinarias.
levels.[i].blocks.[j].mixer.A_log	Parámetro Tensor libre (nn.Parameter)	Formulación matemática que dicta las restricciones diagonales del espacio de estado estable, preservando la convergencia a largo plazo de la señal espacial.
levels.[i].blocks.[j].mixer.D	Parámetro Tensor libre (nn.Parameter)	Componente de salto iterativo escalar asimilado para inyecciones residuales directas durante la ejecución secuencial o computación de escaneo paralelo.
levels.[i].blocks.[j].mixer.out_proj.weight	nn.Linear (Dense)	Contracción del sumatorio ponderado para igualar las dimensiones semánticas del tensor de escape hacia la siguiente capa de la

		macroarquitectura.
levels.[i].blocks.[j].mixer.out_proj.bias	Tensor unidimensional	Sesgo unificador de la salida topológica Mamba.

Escrutinio del Módulo TransformerBlock

Los bloques residuales del espectro posterior de los niveles 3 y 4 emplean una arquitectura de bloque de autoatención Multi-Cabezal (Multi-Head Self-Attention). Las representaciones espaciales transpuestas ingresan a mecanismos que generan iterativamente vectores empaquetados de Consulta (Query), Clave (Key) y Valor (Value), típicamente agrupados bajo la estructura singular qkv.[18]

Prefijo Exacto del state_dict PyTorch	**Instanciación de Clase Subyacente**	**Función Matemática y Dimensionalidad**
levels.[i].blocks.[j].attn.qkv.weight	nn.Linear (Dense)	Proyector aglomerado tridimensional que deriva conjuntamente las representaciones Query, Key y Value.
levels.[i].blocks.[j].attn.qkv.bias	Tensor unidimensional	Sesgo aplicable a las proyecciones conjuntas latentes de atención.
levels.[i].blocks.[j].attn.proj.weight	nn.Linear (Dense)	Unificador proyectivo posterior a la agregación softmax de la puntuación atencional de múltiples cabezales.
levels.[i].blocks.[j].attn.proj.bias	Tensor unidimensional	Vector de desplazamiento residual post-atención.
levels.[i].blocks.[j].mlp.fc1.weight	nn.Linear (Dense)	Extractor multiplicativo del bloque perceptrón multicapa integrado.

levels.[i].blocks.[j].mlp.fc1.bias	Tensor unidimensional	Desplazamiento de activación intermedio del bloque denso.
levels.[i].blocks.[j].mlp.fc2.weight	nn.Linear (Dense)	Contracción de dimensionalidad del bloque generador de representaciones.
levels.[i].blocks.[j].mlp.fc2.bias	Tensor unidimensional	Sesgo terminal previo al sumatorio residual.

Cabezal Global Clasificador (Head)

Después de iterar sobre el complejo entramado de la jerarquía multiescala, el mapa de características tridimensional final es aplanado utilizando una capa de acumulación global iterativa, formulada computacionalmente como un AdaptiveAvgPool2d.[17] El vector lineal final emitido atraviesa una capa de normalización de lotes global de estabilización final y, posteriormente, una interconexión densa final para la predicción del espacio de clases (usualmente correspondiente a los 1000 logit paramétricos del corpus visual de ImageNet).[18]

Las claves del state_dict correspondientes a este desenlace anatómico se muestran a continuación.

Prefijo Exacto del state_dict PyTorch	**Instanciación de Clase Subyacente**	**Función Matemática y Dimensionalidad**
norm.weight	nn.BatchNorm2d (γ)	Escalador universal de las distribuciones pre-activadas.
norm.bias	nn.BatchNorm2d (β)	Desplazamiento aditivo universal previo al cabezal denso.
norm.running_mean	nn.BatchNorm2d (μ)	Esperanza estadística terminal retenida sobre lotes promediados.

norm.running_var	nn.BatchNorm2d (σ^2)	Distribución hiper-variacional terminal capturada.
head.weight	nn.Linear (Dense)	Proyección lineal masiva desde el hiper-espacio dimensional latente a los logit categóricos de clasificación fina.
head.bias	Tensor unidimensional	Ajuste basal aditivo iterativo por cada categoría final predicha. Si se especifica la variable num_classes = 0 (modo de extracción pura de características), este componente se convierte en una identidad morfológica sin componentes extraíbles en el estado.

Fundamentos Matemáticos y Principios de Migración Transdimensional

La arquitectura inter-procedimental de MambaVision expone un desafío infraestructural: la carga de los tensores decodificados directamente desde PyTorch hacia los grafos computacionales constructivos en ecosistemas de Keras 3 impone violaciones morfológicas inherentes. Keras 3 es un marco de nivel superior que exige rigurosidad en la definición direccional de sus operaciones, priorizando el rendimiento optimizado para backends de Álgebra Lineal Acelerada (XLA) y el diseño centrado en canales asimilables (channels-last representation).[15] Por su parte, la especificación en memoria de PyTorch favorece intrínsecamente formatos centrados en el lote de dimensiones precursoras (channels-first) y asume un convenio puramente escalar para el diseño geométrico de matrices transpuestas en proyecciones densas.[27]

La mera inyección programática mediante métodos como target_model.set_weights(torch_weights) sin manipulaciones intermedias degenera irrefutablemente en colapsos algebraicos catastróficos o activaciones distorsionadas sin propagar excepciones semánticas inmediatas, socavando el propósito del rendimiento

multi-entorno.[29] Las desviaciones en las convenciones se explican y mitigan mediante axiomas de transposición topológica estricta documentados iterativamente.

Ecuaciones de Transposición Tensorial para Proyecciones Lineales (Capas Densas)

La capa fundacional nn.Linear de PyTorch especifica el operador matemático primario sobre su vector dimensional de pesos, denotado como W_P. Durante la instanciación teórica abstracta, un proyector de D_{in} características preexistentes a D_{out} características de destino es moldeado como una matriz hiper-espacial bidimensional con el orden subyacente de forma (D_{out}, D_{in}). La ecuación de inferencia para un tensor de activación multivariante X es evaluada formalmente mediante una multiplicación por la transpuesta del estado retenido en memoria más el corolario afín del vector de desviación B_P [30]:

$$Y = XW_P^T + B_P$$

El constructor subyacente del backend de Keras 3 para su entidad abstracta keras.layers.Dense invierte la proposición inicial de alojamiento en memoria, definiendo que la manifestación paramétrica central W_K debe estructurarse intrínsecamente respetando el horizonte direccional de (D_{in}, D_{out}). La operación del núcleo convolucional en el interior del compilador estático XLA omite cualquier transposición computacional extraña iterando en lugar del enfoque aditivo canónico:

$$Y = XW_K + B_K$$

La correlación estricta para lograr consistencia e isomorfismo entre los espacios abstractos de ambos sistemas demanda una reconfiguración previa de inyección. La aserción matemática impone que la matriz proyectada del sistema de destino sea la equivalente transpuesta de su originaria de inferencia [32]:

$$W_K = W_P^T$$

En la construcción de migración computacional operada por NumPy, este principio ineludible se soluciona ejecutando una directiva de ordenamiento explícita sobre los índices axiales: np.transpose(tensor_pytorch, (1, 0)). Los tensores suplementarios encapsulados como descriptores de sesgo (B_P) son estructuras planas carentes de un vector direccional bidimensional y se transmutan a B_K de manera analítica directa sin incurrir en transformaciones espaciales redundantes. Esta norma de transposición subyace no solamente

a los cabezales terminales sino de manera prevalente a las abstracciones paramétricas del SSM: in_proj, x_proj, dt_proj, y out_proj.

Ecuaciones de Alteración de Ejes para Redes Convolucionales 2D (Conv2D)

La disparidad geométrica alcanza su cénit arquitectónico en los dominios del procesamiento de visión multivariada. PyTorch optimiza sus librerías de enlace dinámico CUDA (cuDNN) asumiendo por diseño originario un esquema computacional $NCHW$ (Lote, Canal, Altura, Anchura). La implicación para su bloque instanciable nn.Conv2d genera una especificación de forma estricta que engloba cuatro coordenadas vectoriales indexadas secuencialmente como canales salientes, canales entrantes, y las coordenadas cartesianas espaciales iterativas del campo receptivo: $(C_{out}, C_{in}, K_H, K_W)$.[27]

El paradigma implementado en TensorFlow y Keras adopta universalmente la convención topológica abstracta de $NHWC$ (Lote, Altura, Anchura, Canal). Las representaciones convolutivas subyacentes se ven forzadas a acomodar una disposición radicalmente opuesta para maximizar la lectura en memoria de las estructuras contiguas de la CPU y la GPU. El tensor de ponderación equivalente W_K asume el orden normativo de la morfología espacial primaria seguida incondicionalmente por la conectividad de inter-canales de expansión: $(K_H, K_W, C_{in}, C_{out})$.[26]

La igualación de ambos subespacios de parámetros demanda que el tensor cuadridimensional W_P se permute axiomáticamente mediante un esquema de redistribución cruzada que migre los índices perimetrales a posiciones nucleares y viceversa. Se formula matemáticamente un operador discreto de permutación tensorial $\mathcal{P}$:

$$W_K = \mathcal{P}_{2,3,1,0}(W_P)$$

El operador $\mathcal{P}_{2,3,1,0}$ designa computacionalmente que la coordenada original emplazada en el índice 2 (eje de Altura topológica) avance jerárquicamente a la primera posición (índice 0). Recíprocamente, el índice 3 original (eje de Anchura topológica) asume la segunda iteración de anidamiento (índice 1). La dimensión contenedora del volumen de canales de entrada migra hacia atrás al índice 2, y, finalmente, la profundidad del filtro proyector de salida (originalmente posicionado en la cima geométrica) finaliza el mapeo colapsando a la posición terminal en la jerarquía (índice 3).

El corolario algorítmico explícito invoca la macro funcional np.transpose(tensor_pytorch, (2, 3, 1, 0)) para cada manifestación dimensional paramétrica derivada de los módulos PatchEmbed,

Downsample, y la rama interna total de componentes ConvBlock y sus proyecciones afines incipientes.[28]

Restructuración Dinámica de Convoluciones Temporales Unidimensionales (Conv1D)

La particular hibridación analítica introducida en MambaVision es el abandono parcial del procesamiento denso genérico para incrustar procesadores espaciales agrupados que actúan como operadores atencionales sobre secuencias espaciales aplanadas temporalmente. Estas redes integradas en las abstracciones del MambaVisionMixer (conv1d_x y conv1d_z) procesan sobre la dimensión de la representación secuencial.[9]

Para una matriz unidimensional de tamaño de núcleo expansivo K, nn.Conv1d en PyTorch construye una infraestructura tensorial de tres parámetros ordenada estrictamente bajo el formato (C_{out}, C_{in}, K). Su homólogo natural dentro de Keras 3 altera esta proposición teórica asignando preponderancia computacional secuencial a la dimensión del núcleo, instanciando un campo jerárquico paramétrico moldeado como (K, C_{in}, C_{out}).

La transición lógica del tensor extraído del marco pre-entrenado se somete a una corrección de desplazamiento iterativo que reasigna todas y cada una de las coordenadas cartesianas abstractas:

$$W_K = \mathcal{P}_{2,1,0}(W_P)$$

Traducido, la coordenada de convolución unidireccional adopta un protagonismo inicial, el volumen de entrada se preserva inmutable en el centro hiper-espacial, y el espectro total de profundidad convolucional concluye el modelo representacional en el índice iterativo final. Programáticamente, se aplica np.transpose(tensor_pytorch, (2, 1, 0)) a todo prefijo interceptado que contenga la denominación léxica de convolución agrupada 1D.[32]

Consolidación Secuencial Discreta en Normalización de Lotes (BatchNorm)

Los estabilizadores numéricos conocidos en la literatura científica como Capas de Normalización por Lotes (Batch Normalization) no se sujetan a restricciones de transposición paramétrica dado que sus tensores subyacentes son vectores escalares unidimensionales.[17] La divergencia insidiosa, sin embargo, radica enteramente en el comportamiento metodológico de la Programación Orientada a Objetos inherente en los procesos de carga subyacentes en cada infraestructura (Layer-based model vs. Parameter-based model).[34]

En la manipulación abstracta intra-sesión del flujo continuo, PyTorch dispersa los pesos

normalizadores afines del modelo como entidades autónomas desvinculadas jerárquicamente dentro del state_dict, requiriendo búsquedas léxicas exhaustivas de descriptores secuenciales predefinidos (weight, bias, running_mean, running_var).[34] Keras 3 es estructuralmente dogmático durante la asignación externa en masa de los componentes iterativos pre-entrenados, encapsulando las estadísticas computacionales obligatoriamente a través de un arreglo matricial acoplado de cuatro dimensiones vectoriales agrupadas.[34]

Para una inyección exitosa y libre de colapsos topológicos asimétricos al emplear la función enlazada Layer.set_weights([parametros]), la matriz en PyTorch se empaqueta iterativamente concatenando de manera determinista un orden sub-espacial inmutable de componentes escalares, descrita en la teoría como [gamma, beta, moving_mean, moving_variance], donde gamma equivale axiomáticamente a los pesos afines restrictivos y beta mapea de manera paramétrica lineal al desplazamiento del sesgo escalar inicial.[34]

Transmutación Homóloga de Variables de Estado Mamba (A_log y D)

La naturaleza innovadora del Modelo Lineal Invariante en el Tiempo (SSM) introduce componentes que trascienden el marco normativo de las proyecciones afines multidimensionales canónicas y los núcleos convolucionales paramétricos.[4] Durante la discretización del estado de entrada (Zero-order hold continuous discretization), variables intrínsecas del operador de inferencia analítico se preservan y modifican iterativamente, de las cuales emergen entidades clave como A_log (una restricción representacional logarítmica para conservar autovalores continuos negativos, mitigando inestabilidades explosivas numéricas) y D (el parámetro residual inter-estado de las señales de entrada continuas).[4]

Dada su formulación como vectores incondicionales libres o escalares multidimensionales expandidos carentes de un flujo subespacial iterativo en PyTorch (representados como nn.Parameter independientes explícitamente), la correspondencia entre backends mantiene un isomorfismo total de uno a uno.[18] Los tensores aislados durante la carga estructural carecen de cualquier necesidad axiológica de reorientación matricial bidimensional o espacial. El proceso final implica inyectar las dimensiones idénticas extraídas y procesarlas iterativamente en los contenedores receptivos generados asimilando el mismo paradigma topológico.

Diccionario Analítico de Traducción Léxica y Mapeo Computacional

La convergencia holística y la transposición determinista entre ambos marcos se plasma formalmente en el presente diccionario de traducción computacional. La tabla expuesta detalla sin fisuras los clústeres originarios intra-sistema correspondientes al estado pre-compilado en CUDA, mapeados intrínsecamente a su designación funcional asimilada para una implementación Keras 3 modular robusta, especificando iterativamente la formulación transdimensional aplicada sobre la carga en memoria compartida.

Agrupación de Módulos	Prefijo/Sufijo Origen PyTorch (state_dict_key)	Módulo de Destino Keras 3 (Propuesto)	Estructura Listada Keras y Alteración Transposicional Aplicada
A. PatchEmbed (Incrustador de Tokens Espaciales)	patch_embed.conv_down.0.weight	PatchEmbed.conv_down_0 (Conv2D)	Tensor Único [Kernel]. **Permutación:** np.transpose(T, (2, 3, 1, 0))
	patch_embed.conv_down.1.*	PatchEmbed.norm_1 (BatchNorm2D)	Lista Agrupada [w, b, rm, rv]. **Permutación:** Asimilación escalar inalterada.
	patch_embed.conv_down.3.weight	PatchEmbed.conv_down_3 (Conv2D)	Tensor Único [Kernel]. **Permutación:** np.transpose(T, (2, 3, 1, 0))
	patch_embed.conv_down.4.*	PatchEmbed.norm_4 (BatchNorm2D)	Lista Agrupada [w, b, rm, rv]. **Permutación:** Asimilación escalar inalterada.
B. Módulos de Reducción Jerárquica Inter-Etapa (Downsample)	levels.*.downsample.reduction.0.weight	Downsample.reduction_0 (Conv2D)	Tensor Único [Kernel]. **Permutación:** np.transpose(T, (2, 3, 1, 0))
C. Bloques Residuales Convolucionales Puros (Stages 1-2)	levels.*.blocks.*.conv1.weight & .bias	ConvBlock.conv1 (Conv2D)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (2, 3, 1, 0))

	levels.*.blocks.*.norm1.*	ConvBlock.norm1 (BatchNorm2D)	Lista Agrupada [w, b, rm, rv]. **Permutación:** Ninguna.
	levels.*.blocks.*.conv2.weight & .bias	ConvBlock.conv2 (Conv2D)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (2, 3, 1, 0))
	levels.*.blocks.*.norm2.*	ConvBlock.norm2 (BatchNorm2D)	Lista Agrupada [w, b, rm, rv]. **Permutación:** Ninguna.
D. Bloques del Mezclador de Espacio de Estados (MambaVisionMixer)	levels.*.blocks.*.mixer.in_proj.weight & .bias	MambaMixer.in_proj (Dense)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (1, 0))
	levels.*.blocks.*.mixer.x_proj.weight	MambaMixer.x_proj (Dense)	Tensor Único [Kernel] sin bias. **Permutación del Kernel:** np.transpose(T, (1, 0))
	levels.*.blocks.*.mixer.conv1d_x.weight & .bias	MambaMixer.conv1d_x (Conv1D)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (2, 1, 0))
	levels.*.blocks.*.mixer.conv1d_z.weight & .bias	MambaMixer.conv1d_z (Conv1D)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (2,

			1, 0))
	levels.*.blocks.*.mixer.dt_proj.weight & .bias	MambaMixer.dt_proj (Dense)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (1, 0))
	levels.*.blocks.*.mixer.A_log	MambaMixer.A_log (Layer/Variable)	Variable Directa. **Permutación:** Copia paramétrica asimétrica inalterada.
	levels.*.blocks.*.mixer.D	MambaMixer.D (Layer/Variable)	Variable Directa. **Permutación:** Copia paramétrica asimétrica inalterada.
	levels.*.blocks.*.mixer.out_proj.weight & .bias	MambaMixer.out_proj (Dense)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (1, 0))
E. Cabezal Proyectivo Global de Clasificación Semántica (Head)	norm.*	Model.head_norm (BatchNorm2D)	Lista Agrupada [w, b, rm, rv]. **Permutación:** Ninguna.
	head.weight & .bias	Model.head (Dense)	[kernel, bias]. **Permutación del Kernel:** np.transpose(T, (1, 0))

Algoritmia Estructural y Orquestación Computacional

de la Transferencia

Una simple inferencia de traducción de utilidades abstractas generalistas (como pt2keras) no garantiza fiabilidad empírica debido a la particular naturaleza exótica y acoplada del núcleo iterativo computacional selective_scan_fn.[14] Es imprescindible un proceso asimilativo determinista programado analíticamente paso a paso, abordando las inconsistencias inter-sistemas para asegurar una preservación total de las distribuciones latentes subyacentes sin colapsar el entorno JAX, TensorFlow o Torch agnóstico que gestiona el ecosistema Keras 3.[15]

El algoritmo delineado a continuación se bifurca en un flujo metodológico lógico y cohesivo compuesto por fases consecutivas inmutables:

1. **Fase de Decodificación Cautelosa y Purga Metadática**: Para evitar excepciones perjudiciales de deserialización generadas por hiperparámetros de CUDA subyacentes que puedan no estar presentes en clústeres heterogéneos, se extraen las directivas seriales mapeando universalmente todo objeto a la unidad lógica abstracta (CPU).[16] Con el núcleo consolidado, la limpieza algorítmica itera para suprimir los constructos paralelos de infraestructura multitarjeta, despojando al léxico de cualquier ocurrencia destructiva del infame prefijo module. y de contenedores analíticos prescindibles como iteraciones semánticas encoder..[17] Asimismo, se omiten explícitamente fragmentos contadores estocásticos (num_batches_tracked) dado que envenenan las directrices esperadas de variables de Keras 3 durante el modelado secuencial de lotes.
2. **Fase Heurística Transposicional Continua**: Un motor subyacente de agrupamiento explora heurísticamente las dimensionalidades retenidas por cada subcampo. Basándose en la magnitud tridimensional del vector numérico y analizando sufijos específicos léxicos (.weight asociado a contextos nominales de convolución intraespacial), procesa y aplica la pertinente operación transdimensional formulada mediante una rotación de tensores en memoria continua de NumPy. Paralelamente, empaqueta las constantes de dispersión afín (escalas, desplazamientos, varianzas temporales) para condensarlas sin fisuras dimensionales hacia las construcciones matriciales estáticas pre-empaquetadas.[32]
3. **Fase Condicional Previa a la Ejecución Constructiva Keras 3**: Keras 3, a diferencia de versiones predecesoras abstractas menos astringentes asociadas exclusivamente a backends unificados limitados, castiga rigurosamente las definiciones estructurales diferidas debido al marco universal requerido por JAX. Un intento por sobrescribir hiper-variables tensoriales inexactas antes de procesar el recorrido del grafo completo produce aserciones irrecuperables de NotImplementedError o inestabilidad in-place.[15] De ahí que se ejecute axiomáticamente una iteración ficticia generativa (dummy forward pass) predeterminada antes de asimilar el modelo decodificado.[15]
4. **Fase Top-Down de Ensamblaje Paramétrico Físico**: Habiendo verificado y solidificado exhaustivamente el marco Keras, la infusión se canaliza en bloque recorriendo jerárquicamente las abstracciones léxicas de la clase. Cada tensor es reasignado mediante un mapeado inyectivo inquebrantable empleando el método fundamental .set_weights()

para el compendio genérico, y rutinas aditivas (.assign()) para las métricas invariables del espacio SSM latente.[29]

Implementación del Script de Migración en Python

La síntesis metodológica detallada previamente converge en la siguiente instrumentación de software. Se presupone computacionalmente que las estructuras híbridas de nivel abstracto equivalentes (PatchEmbed, MambaVisionLayer, ConvBlock, MambaVisionMixer, Downsample) han sido instanciadas en un envoltorio primario heredado explícitamente como clase desde keras.Model.

Python

```python
import torch
import numpy as np
import keras
from collections import defaultdict

def decodificar_y_purgar_estado_pytorch(ruta_checkpoint: str) -> dict:
    """
    Rutina de deserialización segura que purga la topología de diccionarios de
    PyTorch de descriptores paralelos irrelevantes para asimilaciones estáticas.
    """
    # Restricción explícita de seguridad analítica forzando carga en memoria general
    checkpoint = torch.load(ruta_checkpoint, map_location='cpu', weights_only=True)

    # Resolver jerarquías de encapsulamiento estocásticas en serialización
    if 'state_dict' in checkpoint:
        estado_diccionario = checkpoint['state_dict']
    elif 'model' in checkpoint:
        estado_diccionario = checkpoint['model']
    else:
        estado_diccionario = checkpoint

    estado_limpio = {}
    for llave_identificadora, tensor_valor in estado_diccionario.items():
        # Extracción y supresión procedimental de artefactos DDP y prefijos base
        llave_procesada = llave_identificadora.replace('module.', '').replace('encoder.', '')

        # Purgado riguroso de iteradores contadores internos asimilables por la lógica
```

```
        if 'num_batches_tracked' in llave_procesada:
            continue

        # Destrucción del grafo dinámico tensor subyacente de Torch
        # para aislar el campo puramente estático de tensores NumPy multidimensionales
        estado_limpio[llave_procesada] = tensor_valor.numpy()

    return estado_limpio

def evaluar_y_mutar_dimensionalidad(llave: str, tensor: np.ndarray) -> np.ndarray:
    """
    Motor heurístico evaluador para transmutar geométricamente las especificaciones
    matrices de PyTorch a los imperativos dogmáticos multidimensionales de Keras 3.
    """
    conteo_dimensional = len(tensor.shape)

    # Regla 1: Alteración Espacial de Convoluciones 2D Puras
    # Formulación transdimensional: (C_out, C_in, H, W) -> (H, W, C_in, C_out)
    if conteo_dimensional == 4:
        return np.transpose(tensor, (2, 3, 1, 0))

    # Regla 2: Alteración Topológica de Convoluciones Secuenciales y Espaciales 1D
    # Formulación transdimensional Mamba Mixer: (C_out, C_in, K) -> (K, C_in, C_out)
    elif conteo_dimensional == 3 and 'conv1d' in llave:
        return np.transpose(tensor, (2, 1, 0))

    # Regla 3: Alteración Bidimensional Aislada de Estructuras Densas Lineales
    # Formulación transdimensional atencional: (D_out, D_in) -> (D_in, D_out)
    elif conteo_dimensional == 2:
        return np.transpose(tensor, (1, 0))

    # Salida nula-iterativa (Tensores de escalado normativo y tensores paramétricos crudos Mamba)
    return tensor

def consolidar_vectores_normalizacion(estado_limpio_mutado: dict) -> dict:
    """
    Unificador asimilador que compendia los vectores de estado de
    BatchNormalization, preensamblándolos orgánicamente.
    """
    diccionario_keras_final = {}
    repositorio_normalizadores = defaultdict(dict)

    for llave_identificadora, tensor in estado_limpio_mutado.items():
        subcampos_evaluadores = ['.weight', '.bias', '.running_mean', '.running_var']
```

```
        # Desglosar condicionalmente estructuras multi-campo
        if any(sufijo in llave_identificadora for sufijo in subcampos_evaluadores):
            prefijo_base, descriptor_sufijo = llave_identificadora.rsplit('.', 1)

            if descriptor_sufijo in ['weight', 'bias', 'running_mean', 'running_var']:
                repositorio_normalizadores[prefijo_base][descriptor_sufijo] =
evaluar_y_mutar_dimensionalidad(llave_identificadora, tensor)
            else:
                diccionario_keras_final[llave_identificadora] =
evaluar_y_mutar_dimensionalidad(llave_identificadora, tensor)
        else:
            # Procesamiento inquebrantable asimilativo de variables no normativas (A_log)
            diccionario_keras_final[llave_identificadora] =
evaluar_y_mutar_dimensionalidad(llave_identificadora, tensor)

    # Formulación del ensamblador:
    for prefijo_base, grupo_componentes in repositorio_normalizadores.items():
        if 'running_mean' in grupo_componentes:
            gamma_escala = grupo_componentes.get('weight',
np.ones_like(grupo_componentes['running_mean']))
            beta_desplazamiento = grupo_componentes.get('bias',
np.zeros_like(grupo_componentes['running_mean']))
            media = grupo_componentes['running_mean']
            varianza = grupo_componentes['running_var']
            diccionario_keras_final[prefijo_base] = [gamma_escala, beta_desplazamiento, media,
varianza]
        else:
            # Reintegrar bloques de red desvinculados de secuencias normativas globales (Densas puras)
            if 'weight' in grupo_componentes:
                diccionario_keras_final[f"{prefijo_base}.weight"] = grupo_componentes['weight']
            if 'bias' in grupo_componentes:
                diccionario_keras_final[f"{prefijo_base}.bias"] = grupo_componentes['bias']

    return diccionario_keras_final

def efectuar_injerto_parametrizado(modelo_compilado_keras: keras.Model, ruta_checkpoint_pytorch:
str):
    """
    Orquestador terminal de la propagación inyectiva de pesos desde la matriz
    en PyTorch hacia los nodos paramétricos del modelo instanciado general en Keras 3.
    """
    # 0. Fase mandataria del motor Keras 3 (Evaluación ficticia forzosa)
    print(" Forzando compilación anticipada e inferencia estructurada de subgrafos...")
```

```
    espectro_tensor_aleatorio = np.random.normal(size=(1, 224, 224, 3)).astype(np.float32)
    _ = modelo_compilado_keras(espectro_tensor_aleatorio)

    print("[1/3] Extrayendo, filtrando lógicamente y esterilizando base analítica...")
    diccionario_base_crudo = decodificar_y_purgar_estado_pytorch(ruta_checkpoint_pytorch)

    print("[2/3] Computando topología de convolución cruzada y resolviendo dimensiones...")
    diccionario_matrices_preparadas =
consolidar_vectores_normalizacion(diccionario_base_crudo)

    print("[3/3] Inyectando campos sinápticos y descriptores continuos asimilativos...")

    # Inyección Nivel 1: Submódulo Stem / Incrustador de Token PatchEmbed

modelo_compilado_keras.patch_embed.conv_down_0.set_weights([diccionario_matrices_prep
aradas['patch_embed.conv_down.0.weight']])

modelo_compilado_keras.patch_embed.norm_1.set_weights(diccionario_matrices_preparadas
['patch_embed.conv_down.1'])

modelo_compilado_keras.patch_embed.conv_down_3.set_weights([diccionario_matrices_prep
aradas['patch_embed.conv_down.3.weight']])

modelo_compilado_keras.patch_embed.norm_4.set_weights(diccionario_matrices_preparadas
['patch_embed.conv_down.4'])

    # Inyección Nivel 2: Módulos Iterativos y Jerarquías Inter-espaciales (Levels)
    for indice_etapa, capa_nivel_actual in enumerate(modelo_compilado_keras.levels):

        # Validación inyectiva direccional para bloques de Submuestreo Reductivo (Downsample)
        if hasattr(capa_nivel_actual, 'downsample') and capa_nivel_actual.downsample is not None:
            llave_referencial_reduccion = f"levels.{indice_etapa}.downsample.reduction.0.weight"

capa_nivel_actual.downsample.reduction_0.set_weights([diccionario_matrices_preparadas[llav
e_referencial_reduccion]])

        # Inyección profunda en el interior iterativo del espectro bloque-a-bloque (Blocks)
        for indice_bloque, estructura_bloque_interno in enumerate(capa_nivel_actual.blocks):
            prefijo_llave_basal = f"levels.{indice_etapa}.blocks.{indice_bloque}"

            # Sub-Condición: Estructuras ConvBlock (Etapas 1 y 2 - Extracción Temprana)
            if hasattr(estructura_bloque_interno, 'conv1'):
                estructura_bloque_interno.conv1.set_weights([
```

```
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.conv1.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.conv1.bias"])

estructura_bloque_interno.norm1.set_weights(diccionario_matrices_preparadas[f"{prefijo_llave_basal}.norm1"])
        estructura_bloque_interno.conv2.set_weights([
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.conv2.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.conv2.bias"])

estructura_bloque_interno.norm2.set_weights(diccionario_matrices_preparadas[f"{prefijo_llave_basal}.norm2"])

    # Sub-Condición: Estructuras Complejas del SSM (Etapas 3 y 4 - Mezclador)
    elif hasattr(estructura_bloque_interno, 'mixer'):
        # 1. Tensores de proyección de rama bidireccional densa
        estructura_bloque_interno.mixer.in_proj.set_weights([
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.in_proj.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.in_proj.bias"])
        # Proyección paramétrica inyectiva sin sesgo generador

estructura_bloque_interno.mixer.x_proj.set_weights([diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.x_proj.weight"]])

        # 2. Tensores dependientes del modelado causal analítico temporal convolucional unidimensional
        estructura_bloque_interno.mixer.conv1d_x.set_weights([
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.conv1d_x.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.conv1d_x.bias"])
        estructura_bloque_interno.mixer.conv1d_z.set_weights([
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.conv1d_z.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.conv1d_z.bias"])

        # 3. Proyecciones terminales hiper-variables latentes
        estructura_bloque_interno.mixer.dt_proj.set_weights([
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.dt_proj.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.dt_proj.bias"])
        estructura_bloque_interno.mixer.out_proj.set_weights([
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.out_proj.weight"],
            diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.out_proj.bias"])

        # 4. Asimilación Continua Directa de Variables Espaciales Sensibles Invariantes
        # Nota Computacional: Utilizar assign() evita reestructuraciones estocásticas de capa iterativa
```

```
estructura_bloque_interno.mixer.A_log.assign(diccionario_matrices_preparadas[f"{prefijo_llave_basal}.mixer.A_log"])
            estructura_bloque_interno.mixer.D.assign(diccionario_matrices_preparadas)

    # Inyección Nivel 3: Consolidador Generalizado Terminal (Clasificador y Cuello de Botella Normativo)
    modelo_compilado_keras.head_norm.set_weights(diccionario_matrices_preparadas['norm'])

    # Restricción Condicional para modelado puramente extractivo (feature extractor backbones)
    if hasattr(modelo_compilado_keras, 'head') and modelo_compilado_keras.head is not None:
        if isinstance(modelo_compilado_keras.head, keras.layers.Dense):
            modelo_compilado_keras.head.set_weights([
                diccionario_matrices_preparadas['head.weight'],
                diccionario_matrices_preparadas['head.bias'])

    print("[✔] Ejecución Culminada: Arquitectura multi-backend poblada asimétrica e isomorficamente.")
```

Consideraciones Analíticas Post-Inyección y Despliegue en Borde

La transmutación de una arquitectura híbrida vanguardista de las especificaciones rigurosas de un laboratorio académico soportado inherentemente por PyTorch a un conducto industrial universal encapsulado bajo Keras 3 es un ejercicio que sobrepasa el trivial intercambio de parámetros; es esencialmente un procedimiento quirúrgico algebraico.[15] Los tensores críticos correspondientes al integrador central dinámico de tiempo continuo en MambaVision —A_log y D— garantizan iterativamente la estabilidad representacional de las funciones subyacentes frente a inyecciones complejas en tiempo de compilación. En el instante preciso de la traducción de variables en un entorno como JAX o un compilador acelerador XLA, cualquier irregularidad bidimensional superpuesta en estos parámetros logarítmicos continuos desembocaría en desestabilización numérica catastrófica de gradientes en subsecuentes ciclos de ajuste fino iterativo (fine-tuning).[9] Al asimilarlos rigurosamente a través de métodos de variable desnuda (.assign()) y no sobrecargas aglomeradas afines (.set_weights()), la infraestructura computacional retiene el paradigma original inalterable.

El imperativo tras este colosal esfuerzo de adaptación reside no en el mero academicismo de validación cruzada, sino en la escalabilidad computacional en ambientes infraestructurales restrictivos.[10] Keras 3 proporciona un pasaje natural hacia infraestructuras perimetrales de IoT de memoria restringida, procesadores tensoriales dedicados (TPUs) de Google, y unificadores de despliegue industrial multiescala al facilitar traducciones automáticas directas desde el código migrado a formatos como TensorFlow Lite o despliegues puros en infraestructuras orientadas a JAX.[15] MambaVision, validado y encapsulado con un injerto anatómico estructuralmente riguroso a través del diccionario transposicional aquí provisto, transciende las

limitantes conceptuales originarias de la representación matricial causal, abriendo vertientes exploratorias sin parangón en el modelado visual generalizado ultraeficiente y altamente escalable.

Obras citadas

1. myscience/mamba: Pytorch (Lightning) implementation of the Mamba model - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/myscience/mamba
2. Mamba: SSM, Theory, and Implementation in Keras and TensorFlow, fecha de acceso: febrero 26, 2026, https://towardsdatascience.com/mamba-ssm-theory-and-implementation-in-keras-and-tensorflow-32d6d4b32546/
3. Here Comes Mamba: The Selective State Space Model | Towards Data Science, fecha de acceso: febrero 26, 2026, https://towardsdatascience.com/here-comes-mamba-the-selective-state-space-model-435e5d17a451/
4. Implementing Mamba in PyTorch - Medium, fecha de acceso: febrero 26, 2026, https://medium.com/@torotoki/getting-started-with-mamba-implementing-mamba-in-pytorch-33d56ccd8393
5. Releases · NVlabs/MambaVision - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/NVlabs/MambaVision/releases
6. [CVPR 2025] Official PyTorch Implementation of MambaVision: A Hybrid Mamba-Transformer Vision Backbone - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/NVlabs/MambaVision
7. [2407.08083] MambaVision: A Hybrid Mamba-Transformer Vision Backbone - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/abs/2407.08083
8. hustvl/Vim: [ICML 2024] Vision Mamba: Efficient Visual Representation Learning with Bidirectional State Space Model - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/hustvl/Vim
9. MambaVision: A Hybrid Mamba-Transformer Vision Backbone - CVF Open Access, fecha de acceso: febrero 26, 2026, https://openaccess.thecvf.com/content/CVPR2025/papers/Hatamizadeh_MambaVision_A_Hybrid_Mamba-Transformer_Vision_Backbone_CVPR_2025_paper.pdf
10. MambaVision: A Hybrid Mamba-Transformer Vision Backbone - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2407.08083v1
11. SAM-Guided Concrete Bridge Damage Segmentation with Mamba–ResNet Hierarchical Fusion Network - MDPI, fecha de acceso: febrero 26, 2026, https://www.mdpi.com/2079-9292/14/8/1497
12. nvidia/MambaVision-L-21K - Hugging Face, fecha de acceso: febrero 26, 2026, https://huggingface.co/nvidia/MambaVision-L-21K
13. A2Mamba: Attention-augmented State Space Models for Visual Recognition - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/LMMMEng/A2Mamba
14. VMamba/vmamba.py at main · MzeroMiko/VMamba - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/MzeroMiko/VMamba/blob/main/vmamba.py

15. Migrating Keras 2 code to multi-backend Keras 3, fecha de acceso: febrero 26, 2026, https://keras.io/guides/migrating_to_keras_3/
16. Saving and Loading Models — PyTorch Tutorials 2.10.0+cu128 documentation, fecha de acceso: febrero 26, 2026, https://docs.pytorch.org/tutorials/beginner/saving_loading_models.html
17. modeling_mambavision.py · nvidia/MambaVision-T-1K at b1de77e17599566d98efb701c0231b1095dc3a67 - Hugging Face, fecha de acceso: febrero 26, 2026, https://huggingface.co/nvidia/MambaVision-T-1K/blame/b1de77e17599566d98efb701c0231b1095dc3a67/modeling_mambavision.py
18. modeling_mambavision.py · nvidia/MambaVision-B-1K at main - Hugging Face, fecha de acceso: febrero 26, 2026, https://huggingface.co/nvidia/MambaVision-B-1K/blob/main/modeling_mambavision.py
19. MambaVision原理和源码调测原创 - CSDN博客, fecha de acceso: febrero 26, 2026, https://blog.csdn.net/cskywit/article/details/142957710
20. modeling_mambavision.py · nvidia/MambaVision-B-21K at ab797f065a7230f493c65bd55b2156660c0b6a40 - Hugging Face, fecha de acceso: febrero 26, 2026, https://huggingface.co/nvidia/MambaVision-B-21K/blame/ab797f065a7230f493c65bd55b2156660c0b6a40/modeling_mambavision.py
21. SoLA-Vision: Fine-grained Layer-wise Linear Softmax Hybrid Attention - arXiv, fecha de acceso: febrero 26, 2026, https://arxiv.org/html/2601.11164v1
22. Mamba Vision - Kaggle, fecha de acceso: febrero 26, 2026, https://www.kaggle.com/code/tngphmvn/mamba-vision
23. Mamba: Linear-Time Sequence Modeling with Selective State Spaces - arXiv.org, fecha de acceso: febrero 26, 2026, https://arxiv.org/abs/2312.00752
24. Completing the Circle: Building a MambaTransformer from Scratch - GoPenAI, fecha de acceso: febrero 26, 2026, https://blog.gopenai.com/completing-the-circle-building-a-mambatransformer-from-scratch-f92a02381bc8
25. Mamba Vision for deepfake detection - Kaggle, fecha de acceso: febrero 26, 2026, https://www.kaggle.com/code/aleksandrpikul222/mamba-vision-for-deepfake-detection
26. Significant differences between Keras/TensorFlow and Torch - vision - PyTorch Forums, fecha de acceso: febrero 26, 2026, https://discuss.pytorch.org/t/significant-differences-between-keras-tensorflow-and-torch/181932
27. Converting a PyTorch Model to Keras: Let's Do It Manually | by Khayrul Islam | Perceptron Perspectives | Medium, fecha de acceso: febrero 26, 2026, https://medium.com/perceptron-perspectives/converting-a-pytorch-model-to-keras-lets-do-it-manually-57b073d309a8
28. Same weights, implementation but different results n Keras and Pytorch - Stack Overflow, fecha de acceso: febrero 26, 2026,

https://stackoverflow.com/questions/66744761/same-weights-implementation-but-different-results-n-keras-and-pytorch
29. Tensorflow Keras Copy Weights From One Model to Another - Stack Overflow, fecha de acceso: febrero 26, 2026, https://stackoverflow.com/questions/48547688/tensorflow-keras-copy-weights-from-one-model-to-another
30. Keras layer weights shape is different compared to other conventions, fecha de acceso: febrero 26, 2026, https://datascience.stackexchange.com/questions/80794/keras-layer-weights-shape-is-different-compared-to-other-conventions
31. Save and Load Your PyTorch Models - MachineLearningMastery.com, fecha de acceso: febrero 26, 2026, https://machinelearningmastery.com/save-and-load-your-pytorch-models/
32. Transferring weights from plantnet pytorch model to keras model, fecha de acceso: febrero 26, 2026, https://discuss.pytorch.org/t/transferring-weights-from-plantnet-pytorch-model-to-keras-model/176144
33. Transferring weights from Keras to PyTorch - Page 2, fecha de acceso: febrero 26, 2026, https://discuss.pytorch.org/t/transferring-weights-from-keras-to-pytorch/9889?page=2
34. How to load weights from Pytorch to Keras layer by layer? - Stack Overflow, fecha de acceso: febrero 26, 2026, https://stackoverflow.com/questions/66715973/how-to-load-weights-from-pytorch-to-keras-layer-by-layer
35. "Missing key(s) in state_dict" error when loading model - Stack Overflow, fecha de acceso: febrero 26, 2026, https://stackoverflow.com/questions/76124928/missing-keys-in-state-dict-error-when-loading-model
36. migrate keras2 code to keras3.0 with pytorch backend, got inplace error · Issue #18895 · keras-team/keras - GitHub, fecha de acceso: febrero 26, 2026, https://github.com/keras-team/keras/issues/18895